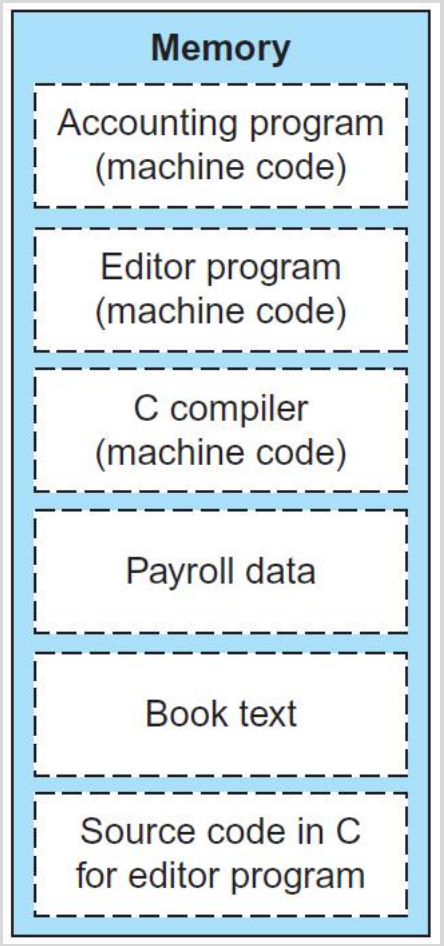
Computer System Architecture

Lecture 3 - Instruction Set Architecture

September 2026

Two Key Principles of Machine Design

1. Instructions are represented as (binary) numbers and, as such, are **indistinguishable** from data.
2. Programs are stored in **alterable memory** (that can be read from or written to) just like data.



Stored-Program Concept

- Programs can be shipped as files of binary numbers – **binary-code compatibility**.
- Computers can inherit ready-made software provided they are compatible with an existing ISA.
- A small number of ISAs.

Assembly Language Instructions

- The language of the machine
  - Goal: make it easy to build the **hardware** and the **compiler** while maximizing **performance** and minimizing **cost**.
- The **MIPS ISA**
  - Microprocessor without Interlocked Pipeline Stages
  - Similar to other ISAs developed since the 1980's
  - Used by Broadcom, Cisco, NEC, Nintendo, Sony, etc.
- Design Goals:
  - Maximize **performance**, minimize **cost**, reduce design time (**time-to-market**), minimize **memory space** (embedded systems), minimize **power consumption** (mobile systems).

CISC vs. RISC

- **Complex Instruction Set Computer** (**CISC**)
  - Lots of instructions of variable size, very memory optimal, typically less registers.
  - Intel x86
- **Reduced Instruction Set Computer** (**RISC**)
  - Instructions, all of a fixed size, more registers, optimized for speed.
  - Usually called a “Load/Store” architecture.
  - MIPS, Sun SPARC, HP PA-RISC, IBM PowerPC

RISC – Reduced Instruction Set Computer

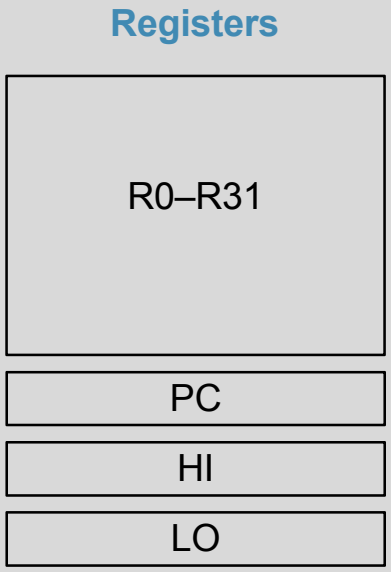
- RISC Philosophy
  - fixed instruction lengths
  - load-store instruction sets
  - limited number of addressing modes
  - limited number of operations
- Instruction sets are measured by how well compilers use them, not by how well assembly language programmers use them.

MIPS (RISC) Design Principles

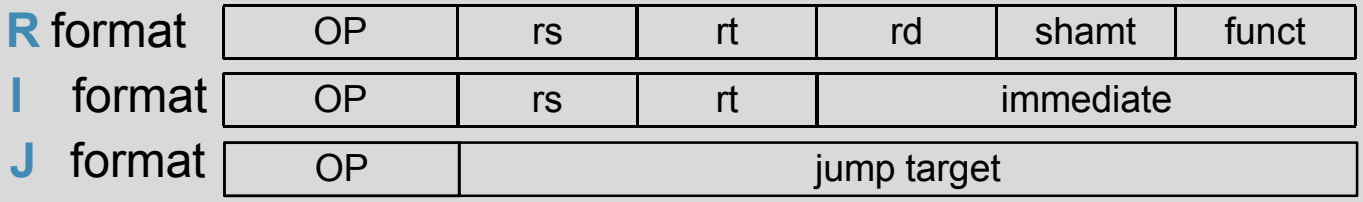
1. **Simplicity favors regularity**
   - fixed size instructions
   - small number of instruction formats
   - opcode always the first 6 bits
2. **Smaller is faster**
   - limited instruction set
   - limited number of registers
   - limited number of addressing modes
3. **Make the common case fast**
   - arithmetic operands from the registers (load-store machine)
   - allow instructions to contain immediate operands
4. **Good design demands good compromises**
   - three instruction formats

MIPS ISA

- Instruction Categories
  - Load/Store
  - Computational
  - Jump and Branch
  - Floating Point
  - Memory Management
  - Special

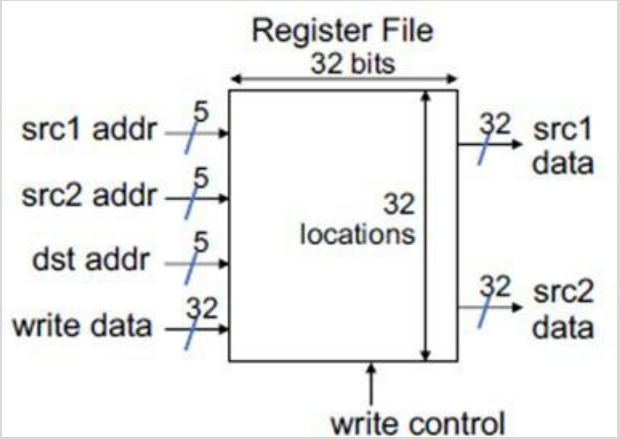


- 3 instruction formats: all **32-bit** wide



MIPS Registers

- Holds thirty-two 32-bit registers
- Two read ports
- One write ports
  - To allow writing, enable the write control bit.



MIPS Registers II

- Registers are
  - **Faster** than main memory
    - But registers with more locations are slower
    - Read/write port increase impacts speed quadratically
  - **Easier** for a complier to use
    - (A*B) – (C*D) – (E*F) can do multiplications in any order
  - Can hold **variables**
    - code density improves

MIPS Register Convention

Name	Register Number	Usage	Preserved on call?
$zero	0	Constant value 0	N/A
$at	1	(Reserved for assembler)	No
$v0–$v1	2–3	Returned values	No
$a0–$a3	4–7	Arguments	No
$t0–$t7	8–15	Temporaries	No
$s0–$s7	16–23	Saved values	Yes
$t8–$t9	24–25	Temporaries	No
$k0–$k1	26–27	(Reserved for OS)	No
$gp	28	Global pointer	Yes
$sp	29	Stack pointer	Yes
$fp	30	Frame pointer	Yes
$ra	31	Return address	Yes

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

```
add a, b, c   # a gets b + c
```

  - All arithmetic operations have this form
- Design Principle 1: **Simplicity favors regularity**
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

Arithmetic Example

- **C code:**

```
f = (g + h) - (i + j);
```

- **Compiled MIPS code:**

```
add t0, g, h          # temp t0 = g + h
add t1, i, j          # temp t1 = i + j
sub f, t0, t1         # f = t0 - t1
```

- **Using registers (f–j are stored in $s0–$s4):**

```
add $t0, $s1, $s2     # $t0 = g + h
add $t1, $s3, $s4     # $t1 = i + j
sub $s0, $t0, $t1     # f = $t0 - $t1
```

Register Operands

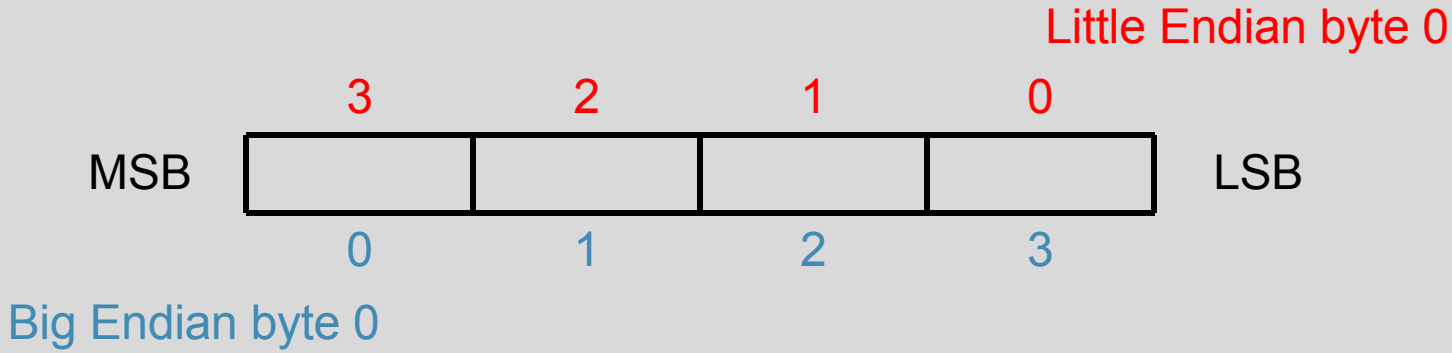
- Arithmetic instructions use register operands
- MIPS has 32 × 32-bit registers
  - Use for frequently accessed data
  - Numbered 0 to 31
  - 32-bit data called a “word”
- Assembler names (convention)
  - $t0, $t1, …, $t9 for temporary values
  - $s0, $s1, …, $s7 for saved variables
- Design Principle 2: **Smaller is faster**
  - (main memory: millions of locations)

Memory Operands

- Main memory used for composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations
  - **Load values from memory into registers**
  - **Store result from register to memory**
- Memory is byte addressed
  - Each address identifies an 8-bit byte
- Words are aligned in memory
  - Address must be a multiple of 4
- MIPS is **Big Endian**
  - Most-significant byte at least address of a word
  - Little Endian: least-significant byte at least address

Big Endian / Little Endian

- **Big Endian**: leftmost byte is word address
  - IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA
- **Little Endian**: rightmost byte is word address
  - Intel 80x86, DEC Vax, DEC Alpha



Memory Operand Example 1

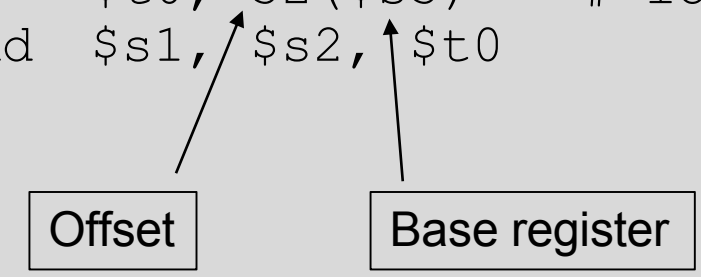
- **C code:**

```
g = h + A[8];
```

  - g in $s1, h in $s2, base address of A in $s3

- **Compiled MIPS code:**
  - Index 8 requires offset of 32 (4 bytes × 8 words)

```
lw   $t0, 32($s3)    # load word
add  $s1, $s2, $t0
```



Memory Operand Example 2

- **C code:**

```
A[12] = h + A[8];
```

  - h in $s2, base address of A in $s3
- **Compiled MIPS code?**

Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important!

Immediate Operands

- Constant data specified in an instruction

```
addi $s3, $s3, 4
```

- No subtract immediate instruction
  - Just use a negative constant

```
addi $s2, $s1, -1
```

- Design Principle 3: **Make the common case fast**
  - Small constants are common
  - Immediate operand avoids a load instruction

The Constant Zero

- MIPS register 0 ($zero) is the constant 0
  - Cannot be overwritten
- Useful for common operations
  - E.g., move between registers

```
add  $t2, $s1, $zero
```

Binary Integers

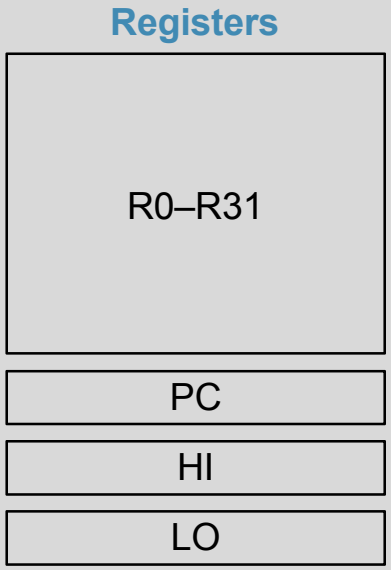
- Unsigned Binary Integers
  - Range: 0 to $+2^n - 1$
  - 32 bit: 0 to +4,294,967,295
- 2's-Complement Signed Integers
  - Range: $-2^{(n-1)}$ to $+2^{(n-1)} - 1$
  - 32 bits: –2,147,483,648 to +2,147,483,647
  - Bit 31 is sign bit: 1 is negative, 0 is positive
  - Some specific numbers
    - 0: 0000 0000 … 0000
    - –1: 1111 1111 … 1111
    - Most-negative: 1000 0000 … 0000
    - Most-positive: 0111 1111 … 1111

Sign Extension

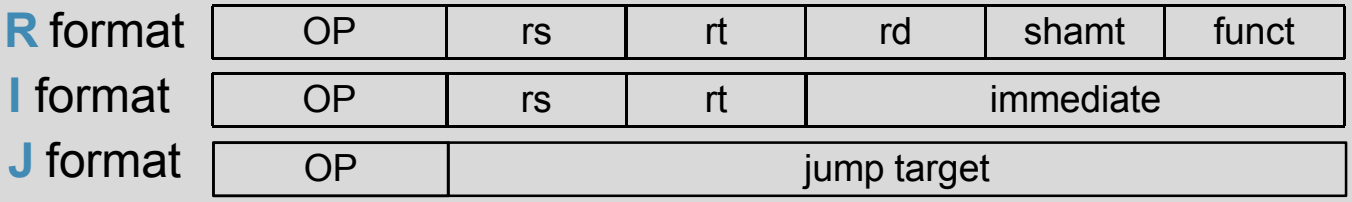
- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb`, `lh`: extend loaded byte/halfword
  - `beq`, `bne`: extend the displacement
- Replicate the sign bit to the left
  - compare unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - –2: 1111 1110 => 1111 1111 1111 1110

MIPS ISA

- Instruction Categories
  - Load/Store
  - Computational
  - Jump and Branch
  - Floating Point
  - Memory Management
  - Special

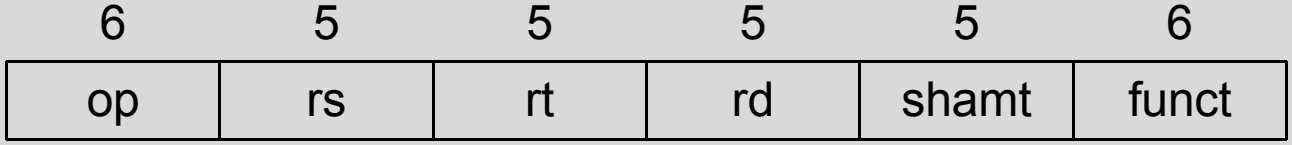


- 3 instruction formats: all **32-bit** wide



MIPS Instruction Fields

- MIPS fields are given names to make them easier to refer to.



- op: 6-bit opcode that specifies the operation
- rs: 5-bit the first register source operand
- rt: 5-bit the second register source operand
- rd: 5-bit the register destination operand
- shamt: 5-bit shift amount
- funct: 6-bit function code that selects specific variant of the operation

R-format Example

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

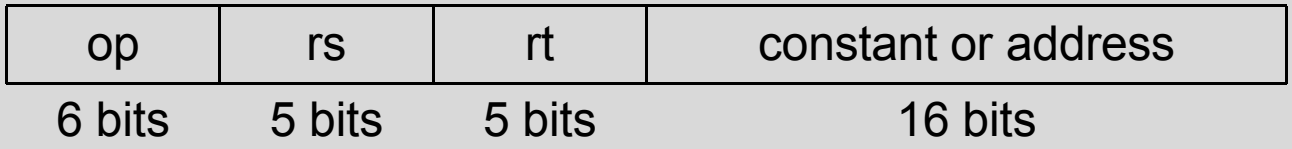
```
add $t0, $s1, $s2
```

$t0 ➔ R8
$s1 ➔ R17
$s2 ➔ R18

0	$s1	$s2	$t0	0	32
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

0000 0010 0011 0010 0100 0000 0010 0000$_2$ = 02324020$_{16}$

MIPS I-format Instructions



- Immediate arithmetic and load/store instructions
  - **rt**: destination or source register number
  - Constant: (-2^{15}) to $(+2^{15}-1)$
  - Address: offset added to base address in **rs**
- Design Principle 4: **Good design demands good compromises**
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

Load/Store (I-format)



6 bits | 5 bits | 5 bits | 16 bits

$t0 ➔ R8
$s3 ➔ R19

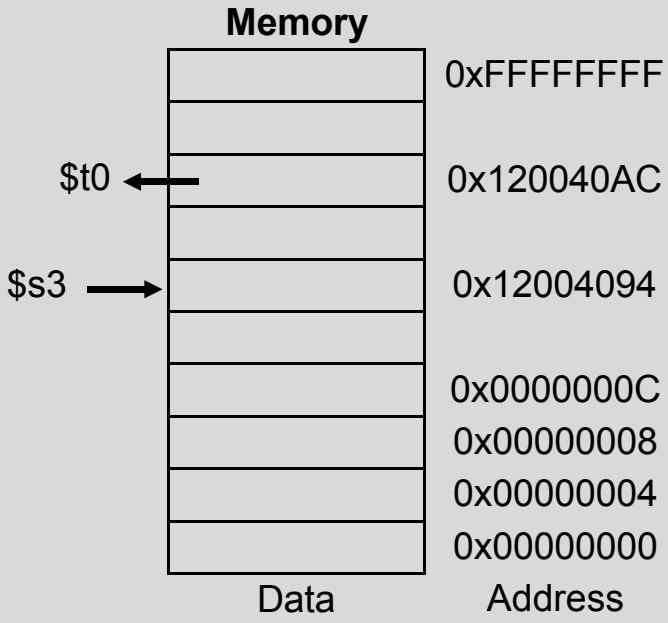
```
lw $t0, 24($s3)
```



$s3 = 0x12004094

$s3 + 24 = 0x12004094 + 0x18
= 0x120040AC

$t0 = data[0x120040AC]



MIPS Instruction Encoding

Instruction	Format	op	rs	rt	rd	shamt	funct	address
`add`	R	0	reg	reg	reg	0	32	
`sub`	R	0	reg	reg	reg	0	34	
`addi`	I	8	reg	reg				constant
`lw`	I	35	reg	reg				address
`sw`	I	43	reg	reg				address

reg: 0–31
constant/address: 16-bit

Translating C Code to Machine Code

- **C code:**

```
A[300] = h + A[300];
```

  - h in $s2, base address of A in$t1
- **Compiled MIPS code?**

- **Machine code?**

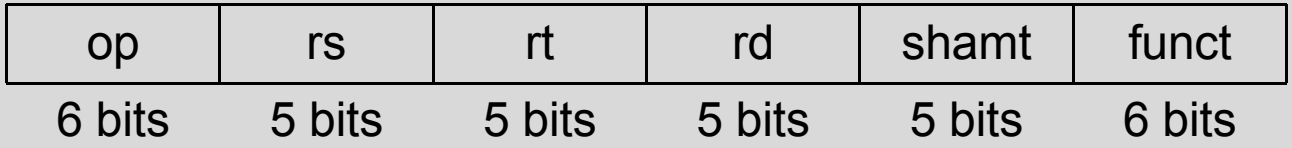
Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	MIPS
Shift Left	<<	<<	`sll`
Shift Right	>>	>>>	`srl`
Bitwise AND	&	&	`and, andi`
Bitwise OR	|	|	`or, ori`
Bitwise NOT	~	~	`nor`

- Useful for extracting and inserting groups of bits in a word

Shift Operations

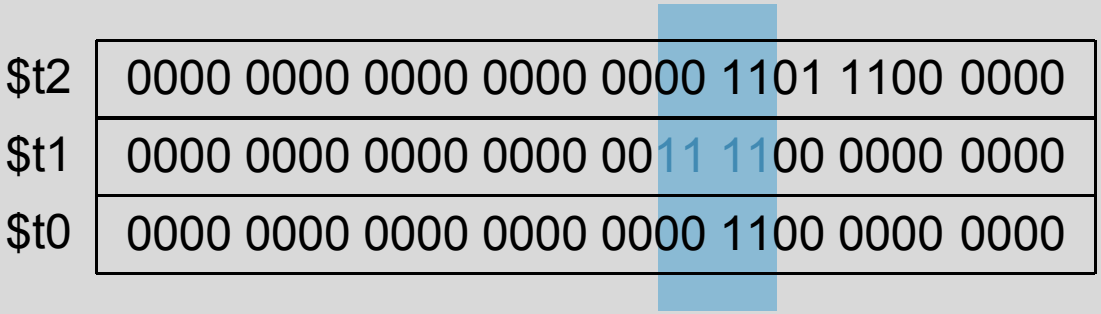


- shamt: how many positions to shift
- Shift Left Logical (`sll`):
  - Shift left and fill with bit 0
  - `sll` by *i* bits equals multiplication by 2^i
- Shift Right Logical (`srl`):
  - Shift right and fill with bit 0
  - `srl` by *i* bits equals division by 2^i (for unsigned numbers)

AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

```
and $t0, $t1, $t2
```

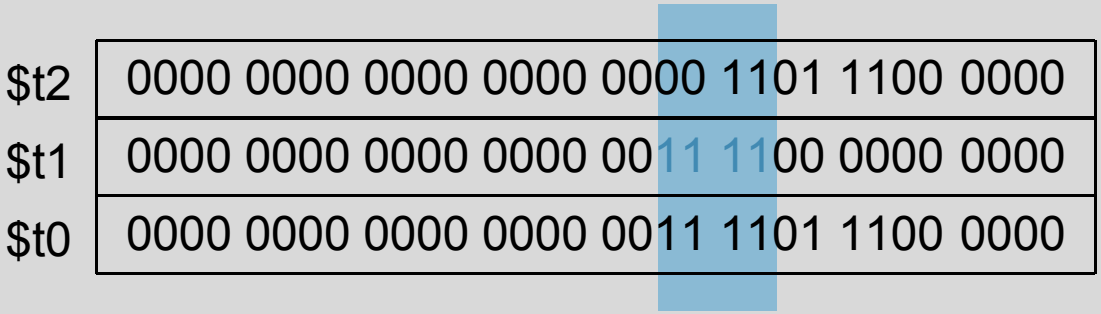


  - What else operation can isolate a field in a word?

OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

```
or $t0, $t1, $t2
```



NOT Operations

- Useful to invert bits in a word
  - Change 0 to 1, and 1 to 0
- NOR: 3-operand instruction
  - a NOR b == NOT (a OR b)

```
nor $t0, $t1, $zero
```

$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	1111 1111 1111 1111 1100 0011 1111 1111

MIPS Logical Operations

- There are a number of bit-wise logical operations in the MIPS ISA
- R-format

```
and   $t0, $t1, $t2     # $t0 = $t1 & $t2
or    $t0, $t1, $t2     # $t0 = $t1 | $t2
nor   $t0, $t1, $t2     # $t0 = not($t1 | $t2)
```

- I-format

```
andi  $t0, $t1, 0xFF00  # $t0 = $t1 & 0x0000FF00
ori   $t0, $t1, 0xFF00  # $t0 = $t1 | 0X0000FF00
```

Conditional Operations

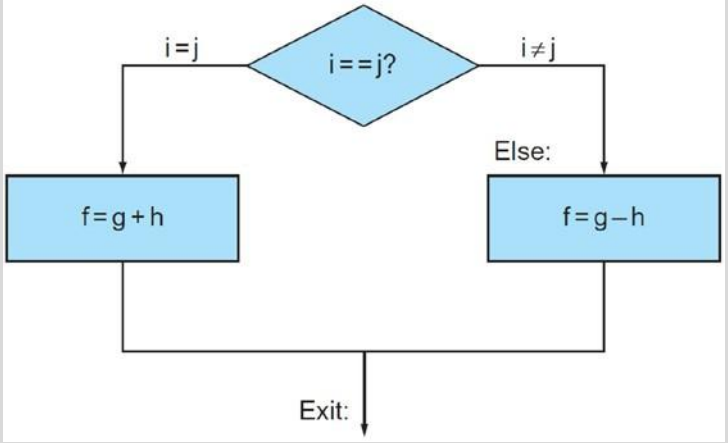
- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- **`beq rs, rt, L1`**
  - if (rs == rt) branch to instruction labeled L1
- **`bne rs, rt, L1`**
  - if (rs != rt) branch to instruction labeled L1
- **`j L1`**
  - unconditional jump to instruction labeled L1

Compiling If Statements

- **C code:**

```
if (i == j) f = g + h;
else f = g – h;
```

  - f in $s0, g in $s1, h in $s2



- **Compiled MIPS code:**

```
      bne $s3, $s4, Else   # go to Else if i≠j
      add $s0, $s1, $s2    # f=g+h (skipped if i≠j)
      j   Exit             # go to Exit
Else: sub $s0, $s1, $s2    # f=g-h (skipped if i=j)
Exit: ...
```

Assembler calculates addresses

Compiling Loop Statements

- **C code:**

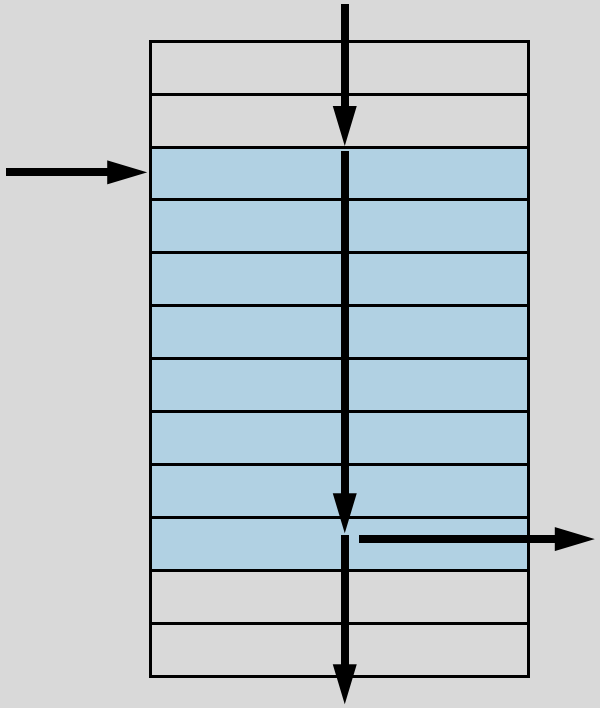
```
while (save[i] == k) i += 1;
```

  - i in $s3, k in $s5, address of save in $s6

- **Compiled MIPS code?**

Basic Blocks

- A **basic block** is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets or branch labels (except at beginning)
- A compiler identifies basic blocks for optimization



More Conditional Operations

- Set result to 1 if a condition is true
  - Otherwise, set to 0
- **`slt rd, rs, rt`**
  - if (rs < rt) rd = 1; else rd = 0;
- **`slti rt, rs, constant`**
  - if (rs < constant) rt = 1; else rt = 0;
- Use in combination with **`beq`**, **`bne`**

```
slt $t0, $s1, $s2     # if ($s1 < $s2)
bne $t0, $zero, L     # branch to L
```

Branch Instruction Design

- Why not **`blt`**, **`bge`**, etc?
- Hardware for <, ≥, … slower than =, ≠
  - Combining with branch involves more work per instruction, requiring a slower clock
  - All instructions penalized!
- **`beq`** and **`bne`** are the common case

Signed vs. Unsigned

- Signed comparison: **`slt`**, **`slti`**
- Unsigned comparison: **`sltu`**, **`sltui`**
- Example:
  - $s0 = 1111 1111 1111 1111 1111 1111 1111 1111
  - $s1 = 0000 0000 0000 0000 0000 0000 0000 0000

```
slt   $t0, $s0, $s1    # signed
```

  - $s0 = −1, $s1 = 0, $s0 < $s1 ➔ $t0 = 1

```
sltu  $t0, $s0, $s1    # unsigned
```

  - $s0 = +4,294,967,295, $s1 = 0, $s0 > $s1 ➔ $t0 = 0

Bounds Check Shortcut

- Treating signed numbers as if they were unsigned gives a low cost way of checking if $0 \leq x < y$ (index out of bounds for arrays)

```
sltu $t0, $s1, $t2      # $t0 = 0 if
                        # $s1 >= $t2 (max)
                        # or $s1 < 0 (min)
beq  $t0, $zero, IOOB # go to IOOB if $t0 = 0
```

- **Negative integers** in two's complement look like large numbers in unsigned notation.
- An unsigned comparison of $x < y$ also checks if x is negative as well as if x is less than y

Procedure Calling

- Caller: the program that instigates a procedure and provides the necessary parameter values.
- Callee: a procedure that executes a series of stored instructions based on parameters provided by the caller and then returns control to the caller.

Procedure Calling

1. Main routine (caller) places parameters in a place where the procedure (callee) can access them
   - $a0–$a3: four argument registers
2. Caller transfers control to the callee
3. Callee acquires the storage resources needed
4. Callee performs the desired task
5. Callee places the result value in a place where the caller can access it
   - $v0–$v1: two value registers for result values
6. Callee returns control to the caller
   - $ra: one return address register to return to the point of origin

Register Usage

- $a0–$a3: arguments (reg 4–7)
- $v0, $v1: result values (reg 2 and 3)
- $t0–$t9: temporaries
  - Can be overwritten by callee
- $s0–$s7: saved
  - Must be saved/restored by callee
- $gp: global pointer for static data (reg 28)
- $sp: stack pointer (reg 29)
- $fp: frame pointer (reg 30)
- $ra: return address (reg 31)

Procedure Call Instructions

- Procedure call: jump and link (**`jal`**)

  `jal ProcedureLabel`

  - Address of following instruction is put in $ra
  - Jumps to target address
- Procedure return: jump register (**`jr`**)

  `jr $ra`

  - Copies $ra to PC
  - PC (program counter): the register containing address of the instruction in the program being executed.

Leaf Procedure Example

- **C code:**

```
int leaf_example (int g, int h, int i, int j)
{
      int f;
      f = (g + h) - (i + j);
      return f;
}
```

  - Arguments g–j in $a0–$a3
  - f in $s0 (hence, need to save $s0 on stack)
  - Result in $v0

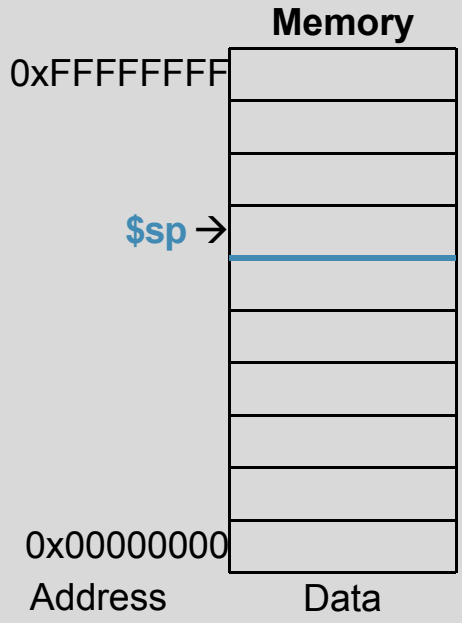
Leaf Procedure Example

- **MIPS code:**

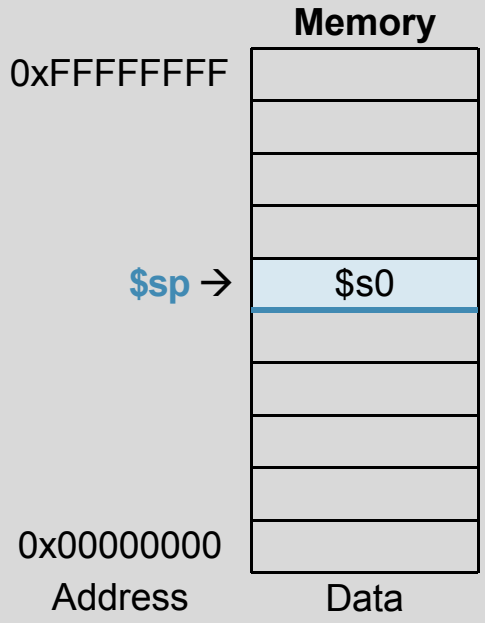
leaf_example:		
addi	$sp, $sp, -4	Save $s0 on stack
sw	$s0, 0($sp)	
add	$t0, $a0, $a1	Procedure body
add	$t1, $a2, $a3	
sub	$s0, $t0, $t1	
add	$v0, $s0, $zero	Result
lw	$s0, 0($sp)	Restore $s0 from stack
addi	$sp, $sp, 4	
jr	$ra	Return

SP (stack pointer): A value denoting the most recently allocated address in a stack that shows where registers should be spilled or where old register values can be found.

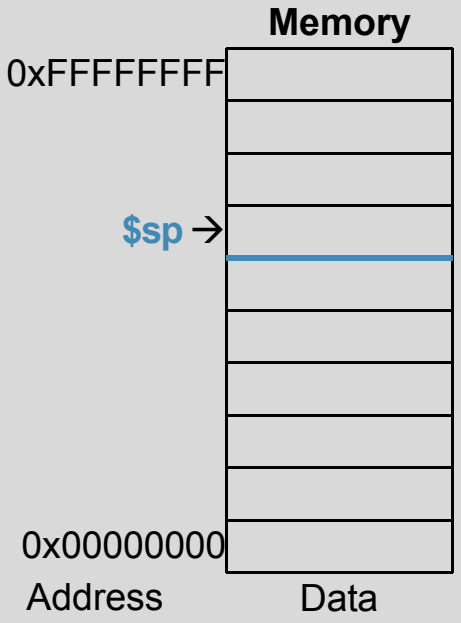
Stack Layout



Before procedure call



During procedure call



After procedure call

SP (stack pointer): A value denoting the most recently allocated address in a stack that shows where registers should be spilled or where old register values can be found.

Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on **stack**:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Non-Leaf Procedure Example

- **C code:**

```
int fact (int n)
{
      if (n < 1) return f;
      else return n * fact(n - 1);
}
```

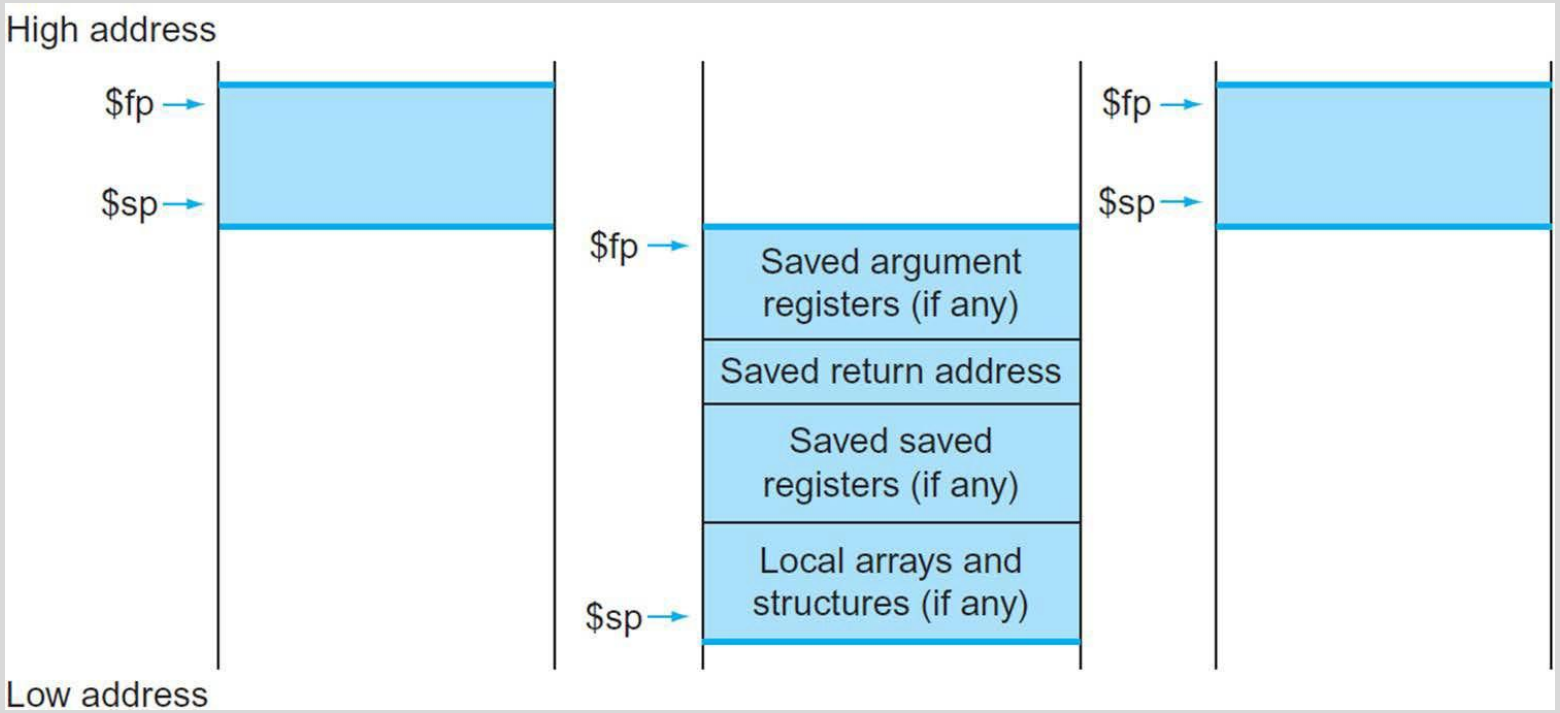
  - Arguments n in $a0
  - Result in $v0

Non-Leaf Procedure Example

- **MIPS code:**

```
fact:
        addi    $sp, $sp, -8        Adjust stack pointer for 2 items
        sw      $ra, 4($sp)         Save return address
        sw      $a0, 0($sp)         Save argument
        slti    $t0, $a0, 1         Test for n < 1
        beq     $t0, $zero, L1
        addi    $v0, $zero, 1       If so, result is 1
        addi    $sp, $sp, 8             pop 2 items from stack
        jr      $ra                     and return
L1:     addi    $a0, $a0, -1        Else decrement n
        jal     fact                Recursive call
        lw      $a0, 0($sp)         Restore original n
        lw      $ra, 4($sp)             and return address
        addi    $sp, $sp, 8         Pop 2 items from stack
        mul     $v0, $a0, $v0       Multiply to get result
        jr      $ra                 And return
```

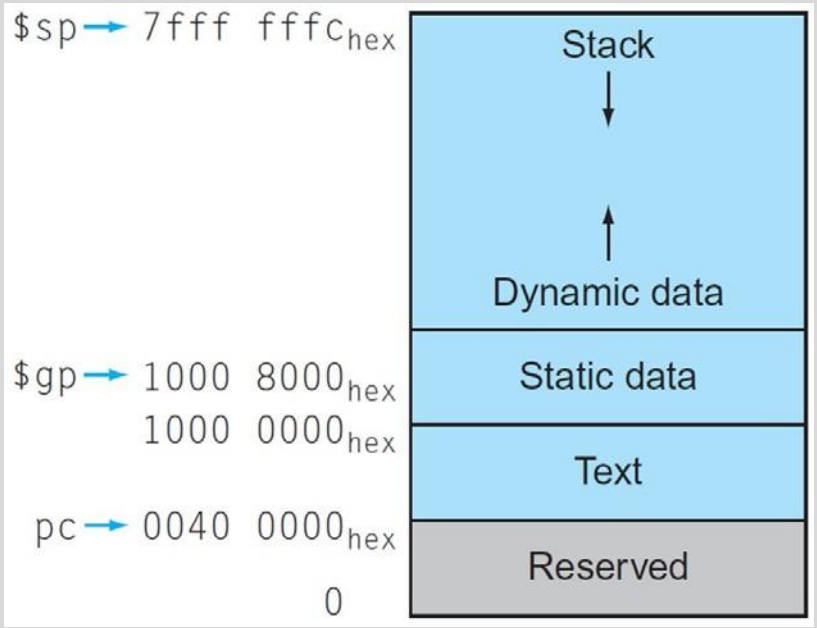
Local Data on the Stack



- Procedure frame (activation record): The segment of the stack containing a procedure's saved registers and local variables.
- FP (frame pointer): A value denoting the location of the saved registers and local variables for a given procedure.

Memory Layout

- **Text**: program code
- **Static data**: global variables
  - e.g., static variables in C, constant arrays and strings
  - $gp initialized to address allowing ±offsets into this segment
- **Dynamic data**: heap
  - e.g., malloc in C, new in Java
- **Stack**: automatic storage



Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Byte/Halfword Operations

- Could use bitwise operations
- MIPS byte/halfword load/store
  - String processing is a common case

```
lb rt, offset(rs)      lh rt, offset(rs)
```

  - Sign extend to 32 bits in rt

```
lbu rt, offset(rs)     lhu rt, offset(rs)
```

  - Zero extend to 32 bits in rt

```
sb rt, offset(rs)      sh rt, offset(rs)
```

  - Store just rightmost byte/halfword

32-bit Constants

- Most constants are small
  - 16-bit immediate is sufficient
- For the occasional 32-bit constant

`lui rt, constant`

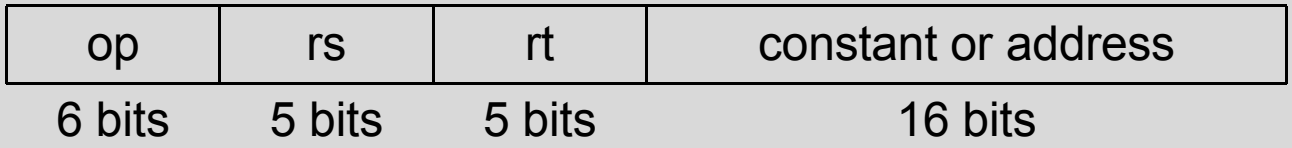
  - Copies 16-bit constant to left (upper) 16 bits of rt
  - Clears right 16 bits of rt to 0

```
lui $s0, 61
ori $s0, $s0, 2304
```

0000 0000 0011 1101	0000 0000 0000 0000
0000 0000 0011 1101	0000 1001 0000 0000

Branch Addressing

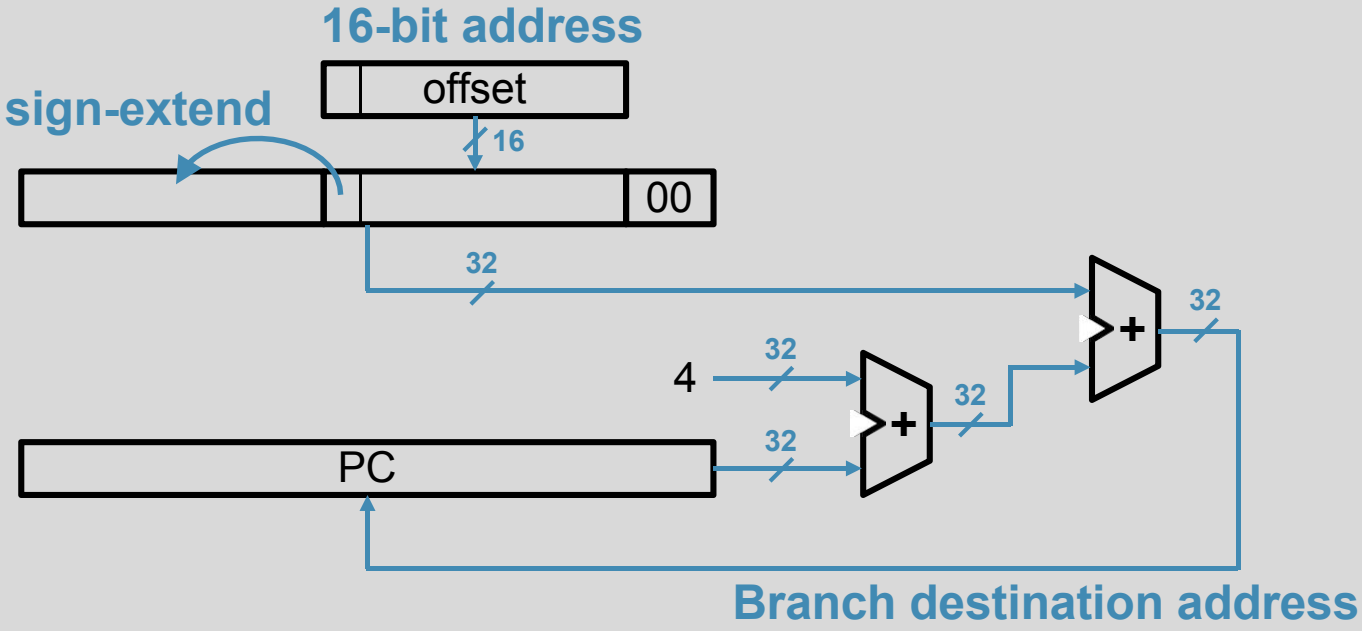
- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward



  - **PC-relative addressing**
    - Target address = PC + offset × 4
    - PC already incremented by 4 by this time

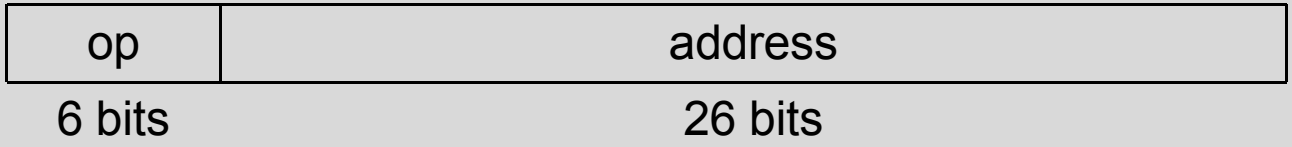
Specifying Branch Destinations

- **Program Counter** (PC) gets updated (PC+4) when fetched.
- Branch distance is limited to -2^{15} to $+2^{15}-1$ (word) instructions away.

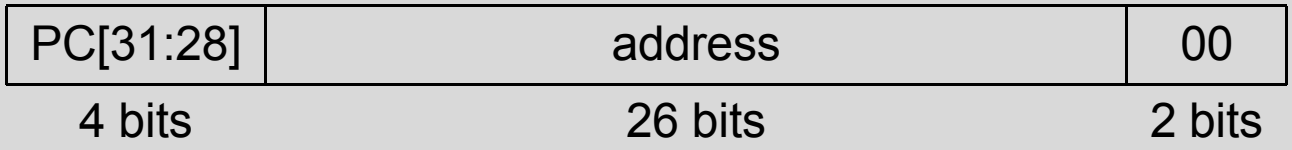


Jump Addressing

- Jump (`j` and `jal`) targets could be anywhere in text segment
  - Encode full address in instruction

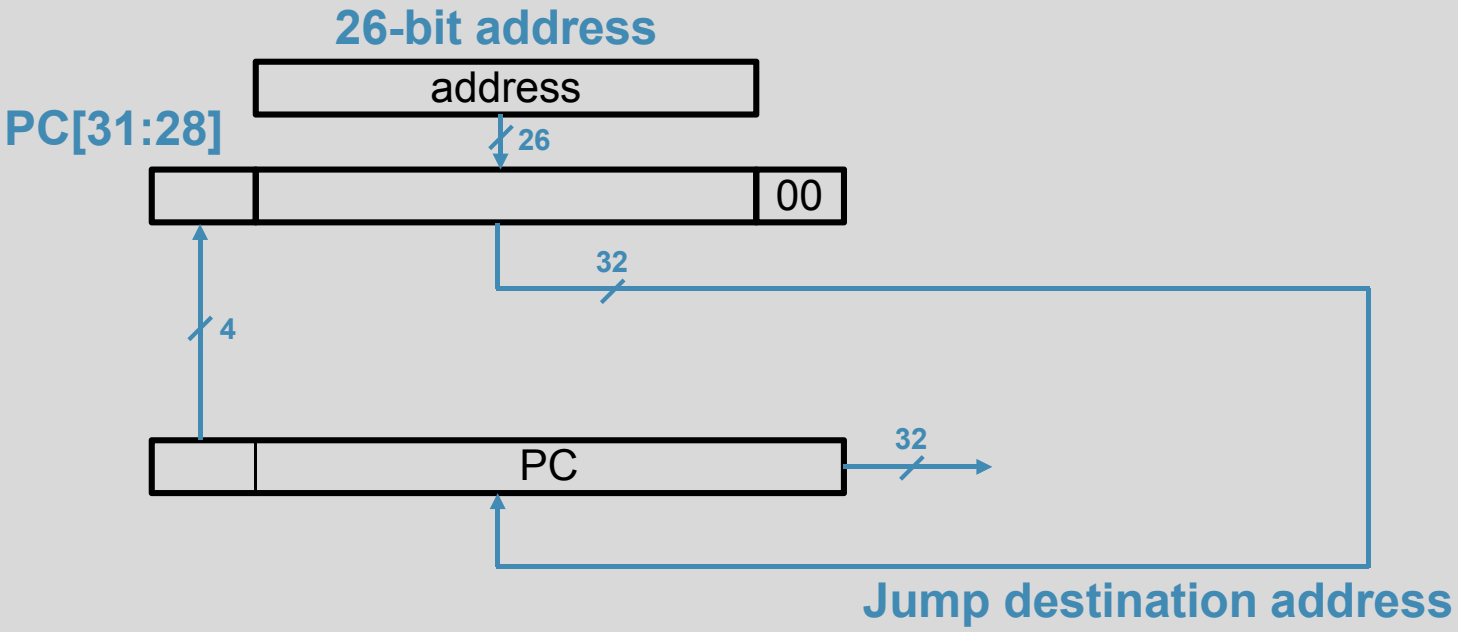


- Pseudodirect jump addressing



Jump Addressing

- Target address must be within the 256-MB region.



Branching Far Away

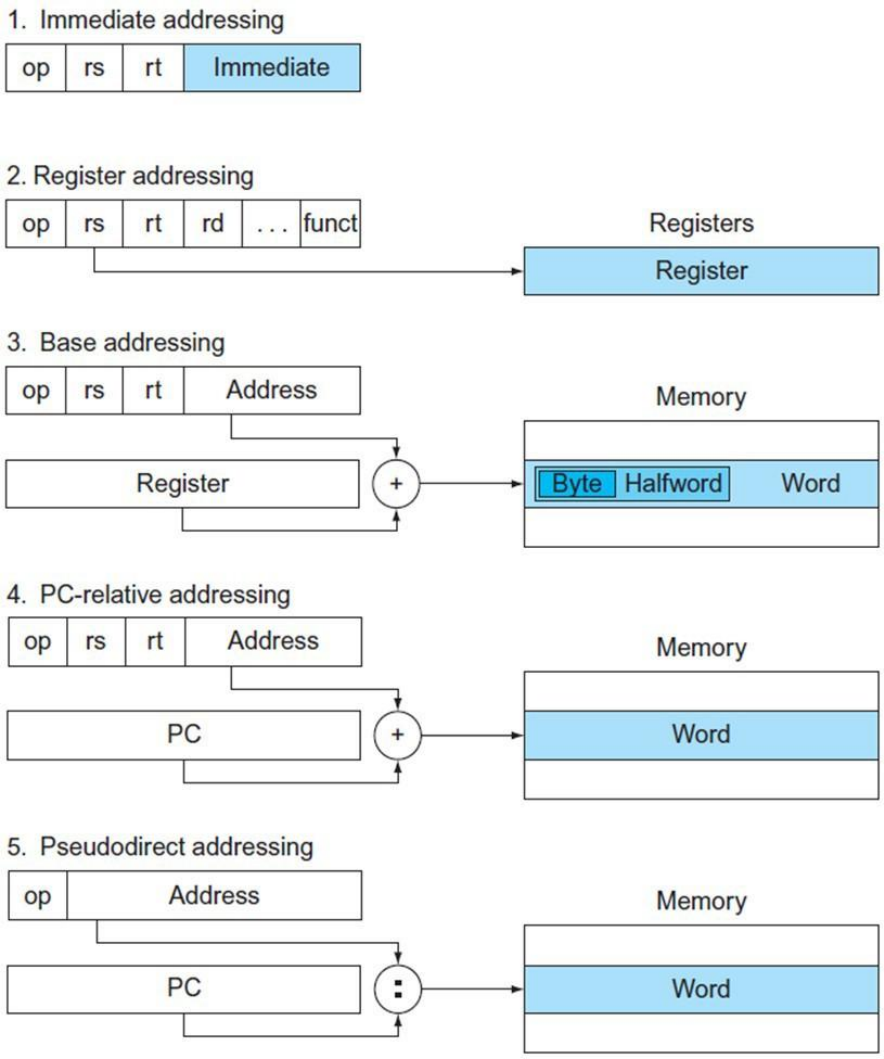
- If branch target is too far to encode with 16-bit offset, assembler rewrites the code:

```
    beq $s0, $s1, L1
```

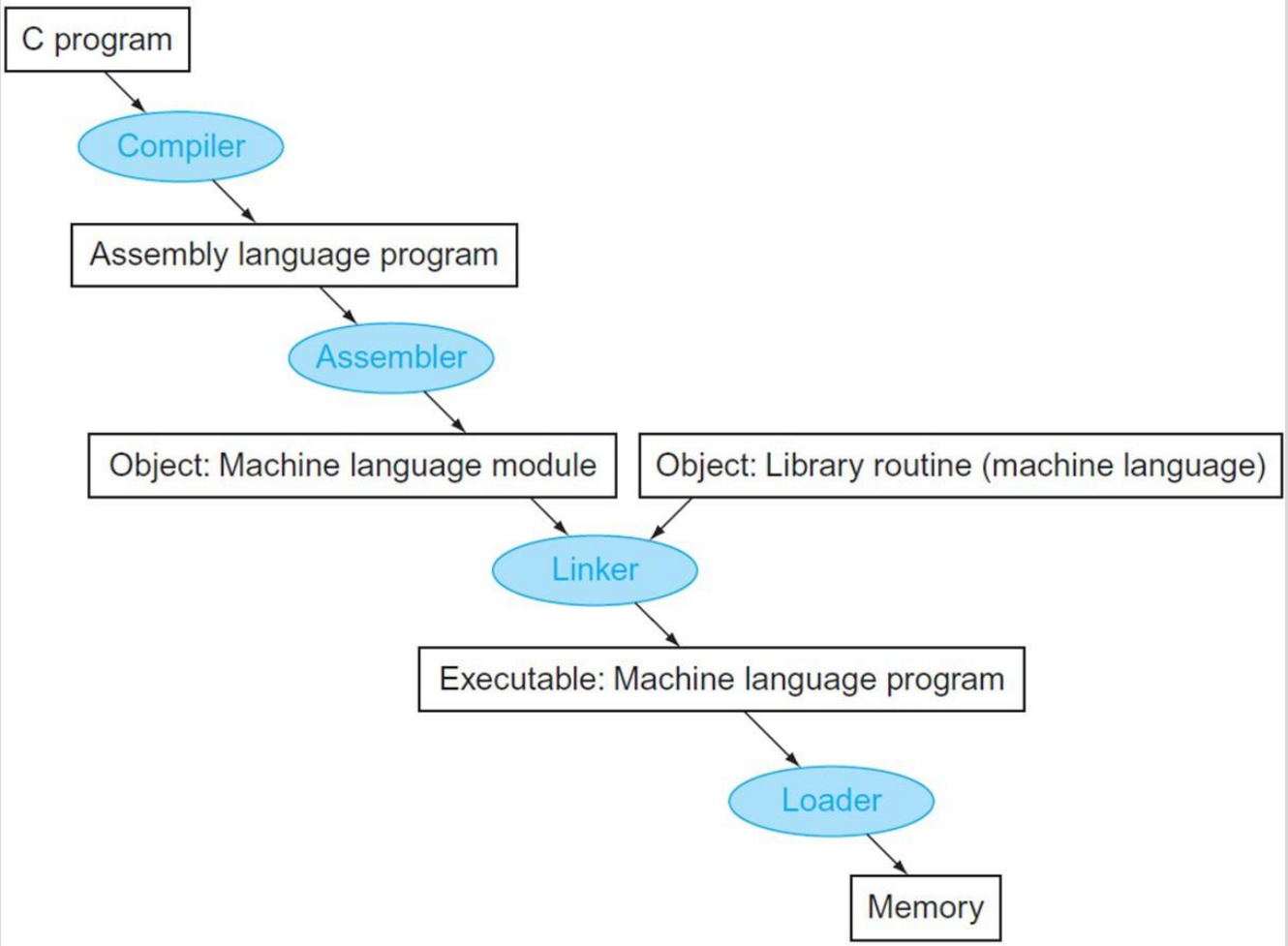
may be replaced by

```
    bne $s0, $s1, L2
    j   L1
L2: ...
```

Addressing Mode Summary



C Code Translation Hierachy



MIPS Organization So Far

